

From an In-Depth Understanding of Multipath TCP Enhancement Schemes to an Adaptive Control Framework in Wireless Networks

Jiangping Han^{ID}, *Member, IEEE*, Yitao Xing^{ID}, Jinhao Liu^{ID}, Kaiping Xue^{ID}, *Senior Member, IEEE*,
Jian Li^{ID}, *Senior Member, IEEE*, David S. L. Wei^{ID}, *Life Senior Member, IEEE*,
Qibin Sun^{ID}, *Fellow, IEEE*, and Jun Lu^{ID}

Abstract—Multipath TCP (MPTCP) has gained popularity to enhance data transmission. From the last decade, proposed MPTCP enhancement schemes for congestion control, path management, and packet scheduling, have been used to benefit transmission performance. However, despite their efforts, they are exigent with a comprehensive understanding of real-world performance to guide the implementation of MPTCP to a more complex wireless network. To that end, we conduct a measurement-driven study of MPTCP enhancement schemes, providing insights and in-depth demonstrations of their performance with a comprehensive real-world platform. Our finding indicates that the enhancement schemes struggle to consistently maintain high performance at all times. One can achieve optimal efficiency in its specific scenarios, but suffers extreme degradation at times. To eliminate this transmission uncertainty in wireless networks, we further propose an adaptive control framework OLSch to integrate different schemes, emphasizing their strengths to provide consistently high performance. To be specific, OLSch is implemented with different scheduling schemes and leverages an online-learning-driven approach to choose one that best fits the current network conditions. Evaluations show that OLSch obviously improves the stability of transmission in harsh network scenarios, eliminates performance degradation, and increases the 95% tail throughput by $1.45\times\text{--}2.39\times$.

Index Terms—Multipath TCP, measurement, wireless networks, congestion control, packet scheduling, online learning.

Received 26 March 2024; revised 17 February 2025 and 25 May 2025; accepted 26 May 2025. Date of publication 9 June 2025; date of current version 7 October 2025. This work is supported in part by the National Natural Science Foundation of China under Grants No. 62302472 and No. 62201540, the Fundamental Research Funds for the Central Universities under grant No. WK2100000039, and Youth Innovation Promotion Association of the Chinese Academy of Sciences (CAS) under Grant No. Y202093. The associate editor coordinating the review of this article and approving it for publication was C. Assi. (*Corresponding author: Kaiping Xue.*)

Jiangping Han, Yitao Xing, Jinhao Liu, Kaiping Xue, Jian Li, and Qibin Sun are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230027, Anhui, China (e-mail: kpxue@ustc.edu.cn).

David S. L. Wei is with the Department of Computer and Information Science, Fordham University, Bronx, NY 10458 USA.

Jun Lu is with the School of Cyber Science and Technology and the Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230027, Anhui, China.

Digital Object Identifier 10.1109/TNSM.2025.3577957

I. INTRODUCTION

MOBILE devices and users are growing rapidly. Nowadays, wireless networks contribute a significant fraction of the Internet traffic [1], making the enhancement transmission service for mobile users over wireless networks an urgent and challenging issue. The critical challenge is, that the low reliability of wireless networks makes the expected transmission quality hard to guarantee. Frequent network failures [2], dynamic network congestion, random packet loss, and other issues make the performance unfulfilled.

Since modern mobile devices are commonly equipped with more than one network interfaces (such as LTE and Wi-Fi), multipath transport protocols [3], [4], [5], [6] can be used to improve the robustness and efficiency of transmissions. Among them, multipath TCP (MPTCP) is the most popular one in wireless scenarios. With simultaneous usage of multiple network paths between peers, MPTCP can offer performance enhancements such as smoother network handover, resilience to network failures, and aggregated bandwidth of multiple network interfaces [7], [8], [9]. And more importantly, MPTCP has been standardized [5] and widely deployed in smartphones [10], [11], [12], upstream Linux kernels [13], and 5G networks [14], because of its effectiveness and applicability to existing applications and networks.

To ensure reliable transmission over multiple paths, MPTCP contains several components, where MPTCP's transmission performance is significantly influenced by its *congestion control*, *path management*, and *packet scheduling* [15], [16]. In the last decade, many enhancement schemes have been proposed for these three components to achieve different transmission objectives. For example, some congestion control schemes are designed to improve TCP's throughput (e.g., BBR [17]) and also bring benefits to MPTCP [18]. Moreover, other congestion control schemes (e.g., LIA [19], OLIA [20]) aim to achieve fairness and resource pool sharing. Other examples are the extensive designs of packet scheduling schemes [21], [22], [23]. They can determine how MPTCP distributes packets among network paths to fulfill different demands, such as to achieve high aggregated throughput (e.g., MinRTT) or high stability (e.g., ReMP [21]).

Although these schemes are well-designed and fine-tuned in their targeted scenarios, not much has been verified in practice

or with universal system-level field tests. **Can MPTCP always work at its best with some specific schemes? If not, which schemes are recommended and what is the reasoning behind these recommendations?** This paper answers all these questions with an in-depth measurement-driven study on MPTCP in wireless networks. Our study comprehensively demonstrates the performance of the existing schemes and provides unique insights, which not only guide the deployment of MPTCP but also show the reasons for performance degradation in wireless networks. Based on the findings, we further propose an adaptive framework that significantly achieves the high effectiveness of MPTCP in wireless networks.

The main contributions of this paper are as follows:

To the best of our knowledge, this paper presents the first in-depth, practical evaluations of crucial schemes in MPTCP implementation. Our study features a large number of real-world measurements and thorough evaluations on MPTCP's congestion control, path management, and packet scheduling schemes in typical wireless network scenarios with various workloads. To support the experiments, we implement an extended MPTCP-enabled kernel with additional improvements and functions. This kernel provides new options for MPTCP connections and enriches the perspectives to analyze the pros and cons of each scheme. We further build an experiment control system to conduct long-term, effective, and controlled measurements. This system provides automated experiment setups and detailed data collection without human intervention. Results of over 240,000 measurements are collected and analyzed to thoroughly understand MPTCP's behaviors and further improve its performance in wireless networks.

Our study provides unique insights into the deployment of different schemes in MPTCP, answering the question that which schemes should be the best. We provide a comprehensive demonstration of MPTCP's performance with different schemes in various scenarios, which can be a factual basis for the deployment and development of MPTCP. We clearly illustrate the suitable use cases where the schemes perform well and the inappropriate use cases where they conversely decline the overall performance. In general, it is observed that existing schemes each have their strengths, yet none of the rigid schemes can perform optimally and stably in the ever-changing conditions of wireless networks. This variability significantly reduces the efficiency of MPTCP. We also find that packet scheduling schemes have a greater impact on performance. However, based on performance measurement of various MPTCP component combinations, we recognize that the interaction effects between congestion control and scheduling cannot be ignored. This complexity makes it challenging to design a scheduling algorithm that adapts to all congestion control schemes.

We propose an adaptive control framework to integrate MPTCP enhancement schemes and bring flexibility and adaptiveness to ever-changing wireless networks. Designing a new scheme from scratch is always time-consuming and labor-intensive, whereas integrating existing schemes can leverage their strengths and compensate for their weaknesses, leading to the best possible performance. We provide an

online-learning-driven framework OLSch that can integrate effective schemes and dynamically use the optimal one for the current network conditions. Then it leverages the advantageous features of integrated strategies and avoids the performance degradation caused by inappropriate use cases. In our implementation, OLSch integrates different scheduling schemes, because our evaluations prove that scheduling schemes provide the most influence in wireless networks. OLSch is implemented as a kernel module and can be a drop-in replacement for in-kernel packet scheduling schemes. Experimental evaluations show that OLSch can provide quality-guaranteed transmission in various network scenarios with high reliability. Specifically, practical evaluations show that, existing in-kernel scheduling schemes suffer throughput loss over 20% of the maximum single path throughput in beyond 30% of the experiments, while OLSch eliminates such degradation and increases the worst-case throughput (i.e., the 95% tail throughput) by 1.45 to 2.39 times.

The rest of this paper is organized as follows. Section II includes an overview of MPTCP and the necessity of measurement study. Section III presents the design of our measurement platform and the experimental setup. In Section IV, we present insights into the MPTCP schemes, followed by the experimental results and related analysis. Based on the findings, we propose an improvement and present its evaluation in Section V. Finally, we conclude the paper in Section VI.

II. BACKGROUND AND RELATED WORKS

In this section, we first present an overview of MPTCP and its three crucial functioning components. Then, we explain why a measurement study on these components is needed.

A. MPTCP Overview

Multipath TCP (MPTCP) enables a transport-layer connection to operate simultaneously across multiple network paths (a.k.a. subflows). Using multiple subflows concurrently, MPTCP can achieve higher throughput [20], [24] and enhanced resilience to network handovers or failures [8], [9]. Besides, MPTCP, as a standardized extension of regular TCP [5], is transparent to both upper and lower layers, which makes it applicable to existing applications and networks. MPTCP has been adopted by some major smartphone vendors such as Apple, Samsung, and Huawei [10], [11], [12], supported by upstream Linux kernels [13], and has become a part of future 5G networks [14]. To improve its performance, researchers have extensively studied congestion control, path management, and packet scheduling in MPTCP. These three components play crucial roles in the multipath transmission process.

B. Congestion Control

Congestion control is a mechanism that controls the entry of data packets into the network, aiming at better use of shared network infrastructures and avoiding congestive collapse. TCP uses congestion control schemes (CCs) to control the transmission rate of data packets, such as Reno, Cubic [25] and BBR [17]. These schemes can also be used in MPTCP

because each MPTCP subflow can be treated as a separate TCP connection. There are also CCs typically designed for MPTCP: Wischik et al. proposed a coupled congestion control (named LIA) [19] for MPTCP to increase the overall efficiency of the network and to achieve fairness to single-path TCP (SPTCP) [26]. Then, opportunistic LIA (OLIA) [20], and balanced linked adaptation (BALIA) [27] are proposed to strike a balance of an inevitable tradeoff among TCP-friendliness, responsiveness, and window oscillation.

C. Path Management

Path management scheme controls the establishment of new subflows. Generally, the default Fullmesh should be used in most deployments [15]. Fullmesh advertises the IP addresses of a host to the remote peer and listen to the IP addresses from the remote one. A typical use case for this Fullmesh path management scheme is a dual-homed client connected to a single-homed server (e.g., a smartphone connected to a regular server). In this case, two subflows will be established on each interface (i.e., LTE and Wi-Fi) of the dual-homed client. In practical use, a significant part of path management is the choice of the primary subflow (i.e., the first subflow established). The network conditions of the primary subflow can largely influence the overall transmission performance, especially for short flows [28]. As a result, we mainly focus on the performance of different primary subflow selections in this paper.

D. Packet Scheduling

Packet scheduling schemes determine how MPTCP distributes packets among the subflows. The default scheduling scheme of the MPTCP kernel [29], named MinRTT, sends packets to the subflow with the smallest round-trip-time (RTT) among all active subflows with an open congestion window. Except for MinRTT, many scheduling schemes with different design objectives have been proposed. For example, the ReMP [21] scheduling scheme (a.k.a. the redundant scheduling scheme) sends packets redundantly on all available subflows to achieve the lowest possible transmission delay and enhance the resilience to subflow failures at the cost of declined throughput.

Despite the above-mentioned schemes, many packet scheduling schemes are proposed to fulfill different objectives. BLEST [22] and ECF [23] are designed to avoid the HoL blocking issue in highly heterogeneous networks. And some other scheduling schemes [30], [31], [32], [33] send packets in a way to avoid out-of-order delivery. These well-designed scheduling schemes may lose their effectiveness in wireless networks because they rely on measurements of the network conditions (e.g., throughput, RTT, etc.), which can be lagging and imprecise in wireless scenarios.

Further, machine-learning empowering scheduling schemes can generate adaptive scheduling strategies for different network scenarios. ReLeS [34] and EdAR [35] apply DQN to generate scheduling strategies with data collected from each finished MPTCP connection. They can achieve great performance with a well-trained neural network. Peekaboo

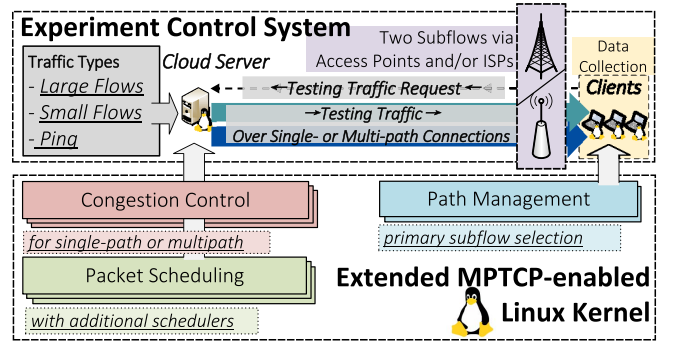


Fig. 1. The overview of our measurement platform.

is an online-learning-based scheduling scheme [36] designed for MPQUIC. It decides whether to use another subflow or not when the fastest subflow is window-limited by leveraging LinUCB algorithm [37] in contextual MAB theory [38].

E. How do the Schemes Perform in MPTCP?

Though the schemes mentioned above have been evaluated in their related papers, we still know little about their actual performance in real wireless networks, especially for the schemes with different designs and objectives. To understand their performance and make better use of them, we need to measure the actual performance of different MPTCP schemes in real wireless networks.

To promote the deployment of MPTCP, many measurement-based studies have been done to evaluate its performance. Raiciu et al. [16] reported their experience in implementing MPTCP in the Linux kernel and evaluated its performance in wireless networks with Wi-Fi and 3G interfaces. Li et al. [9] performed a comprehensive large-scale measurement on MPTCP in high-speed mobility scenarios. In [39], [40], [41], researchers explored the performance of MPTCP in wireless networks, proving its robustness in mobile networks and resilience to network handovers. These measurement-based studies brought valuable insights into MPTCP's performance in the wild, but the schemes for congestion control, packet scheduling, and path managing remain unexplored. To fill the gap, we present an in-depth measurement-driven study that focuses on comparing various schemes of the three components of MPTCP in wireless networks. With the insights and analysis presented in the study, we can understand more about the performance of these schemes in the wild and further guide the practical deployment for the further improvement of MPTCP.

III. MEASUREMENT PLATFORM DESIGN AND EXPERIMENTAL SETUP

This section describes the design of the platform and presents the details of the measurements. It also includes a brief summary of all the schemes used in the measurements.

A. The Extended MPTCP Linux Kernel

Our platform includes an extended MPTCP-enabled kernel and an experiment control system as shown in Fig. 1. To

TABLE I
MPTCP SCHEMES INCLUDED THE MEASUREMENT PLATFORM

Components	Schemes	Brief Descriptions
Congestion Control	BBR	Model-based, modeling the network with the maximum bandwidth and round-trip time. [17]
	CUBIC	Loss-based, the congestion window is a cubic function of time.
	(New) Reno	Loss-based, classic AIMD scheme, Slow Start + AIMD + Fast Retransmit + Fast Recovery.
	LIA	Multipath, loss-based, slowing down the increase of the window for friendliness to SPTCP, the default for MPTCP. [26]
	OLIA	Multipath, loss-based, advanced responsiveness and congestion balancing compared to LIA. [20]
	BALIA	Multipath, loss-based, balancing tradeoff between friendliness and responsiveness. [27]
Packet Scheduling	wVegas	Multipath, delay-based, using packet queuing delay as congestion signals, achieving fine-grained load balancing. [42]
	MinRTT	Always sending packets to the subflow with the lowest RTT among the subflows with open congestion windows.
	RR (Round-Robin)	Sending packets to the subflows with open congestion windows in a round-robin fashion.
	ReMP	Sending each packet redundantly on all available subflows. [21]
	*ECF	Skipping slower subflows if sending data on them takes more time than waiting for the fastest subflow to be available. [23]
	BLEST	Skipping slower subflows if they consume the send window and block the fastest subflow from using it. [22]
Path Management	*OLSch	Always using the optimal scheduling scheme according to current network conditions (Section V).
	Fullmesh	Establishing subflow over all possible IP address peers between two hosts.
	*Primary Subflow Selection	Deciding which network interface should be used to establish the first subflow.

evaluate congestion control, path management, and packet scheduling schemes in real networks, an MPTCP-enabled kernel is needed.

Our extended kernel builds on the MPTCP v0.95 Linux kernel [29], chosen for its stability and broad support. We have expanded the capabilities of the MPTCP kernel and introduced an experiment control system. The enhancements and modifications include:

Enhanced Path Scheduling: We introduce additional schemes to the MPTCP Kernel, beyond the original path scheduling schemes. The kernel originally included several schemes. We also integrate ECF [23], modifying its implementation from MPTCP v0.89 to be compatible with MPTCP v0.95.

Path Management: The existing MPTCP implementation lacks a mechanism to determine the optimal interface for the primary subflow to achieve the best transmission performance. Traditionally, a host selects the interface at the top of the routing table to establish a connection, and the primary subflow operates over this interface. We implement a function that adjusts the metrics of network interfaces to reorder the routing table, facilitating our experiments with various primary subflow choices.

Experiment Control System: We develop a system to manage network interfaces and transmission protocols, alter schemes used in the kernel, and set experimental parameters for each test.

Table I lists all schemes used in this paper. The extended contents are marked with *.

B. The Experiment Control System

The experiment control system controls the behaviors of all hosts to conduct numerous experiments and collect the experimental results. The system includes client-side and server-side software. By leveraging tools such as `ifconfig`, `sysctl`, `ifmetric` and `nmcli`, it can manage the network interfaces to select the primary subflow, choose to use MPTCP or single-path TCP, and change different schemes for congestion control, path management and packet scheduling. For each experiment, the system generates specific-sized traffic between a client and a cloud server. Then it records the download time, the bytes received from each subflow and other detailed information as experimental results.

TABLE II
NETWORK ENVIRONMENTS OF THE CLIENTS

Location	WLAN	LTE
Office	ISP-2	ISP-1
Corridor	Public AP	ISP-1
Dorm	Private AP	ISP-1

Further, the system determines the experimental settings for each experiment, including the flow size, the schemes for the three components, etc. To make sure that the randomness in wireless networks does not cause biased results on certain comparison schemes, the system distributes experiments of different settings evenly and randomly across the lifetime of our measurements. We give detailed settings in Section IV when we present the related results. Besides, the results have been double-checked by the system before we get them to ensure the correctness of our analysis. For example, the system excludes the experiments where MPTCP subflows are not established successfully or the results are abnormally low due to network interface failures by reviewing the raw data.

C. The Network Environments

We deploy several servers on the cloud. And our clients are carefully placed in three locations under typical network scenarios. They get access to the Internet via three most common access methods, namely public access points (APs), private APs, and cellular networks (shown in Table II). Each client is equipped with two network interfaces, which are the built-in Wi-Fi module and a USB cellular modem for LTE. The client in our office (denoted by Office in the rest of the paper) gets access to the Internet via two cellular networks ISP-1 and ISP-2, with an additional USB cellular modem as a Wi-Fi hotspot. The client outside our office in the corridor (denoted by Corridor) uses the public *eduroam* AP as well as an ISP-1 cellular network. The last one is in a dormitory (denoted by Dorm), and it uses an ISP-1 cellular network and a private AP with a rate control policy. These network scenarios are very common in everyday mobile device usage, and thus we choose to evaluate the MPTCP schemes in these environments.

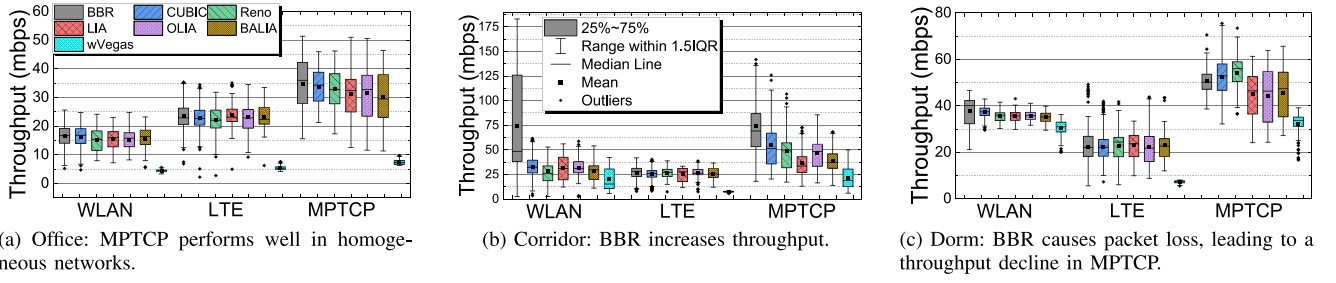


Fig. 2. MPTCP's throughput of different congestion control schemes.

TABLE III
THE MEAN (μ) AND STANDARD DEVIATION (σ) OF THE COMPLETION TIME (MS) OF *Ping* TEST

Path	Office		Corridor		Dorm	
	μ	σ	μ	σ	μ	σ
LTE	43.44	5.62	37.88	1.16	35.11	1.45
WLAN	40.23	3.95	20.81	7.89	19.75	5.23

IV. MEASUREMENTS AND INSIGHTS

In this section, we present the results of our large-scale measurements. By analyzing the results, we further present some unique insights and elucidate the justification. This section includes 4 parts. We focus on studying MPTCP's congestion control, path management, and packet scheduling in the first three parts. Then in the last part, we illustrate a severe drawback of existing MPTCP scheduling schemes. We have conducted 245,358 experiments in total, 48,814 in the Office, 92,360 in the Corridor, and 104,184 in the Dorm.

First of all, we present the results of the basic *Ping* test. The *Ping* test records the time from sending a request for a 10B file to receiving the response. Basically, it is used to test the application level round-trip time (RTT) between clients and the cloud server, which represents the transmission delay for each network path. The mean and standard deviation of the results are shown in Table III. Note that the completion time does not include the three-way handshake time of TCP connection establishment.

A. Congestion Control

Firstly, we compare the performance of different congestion control schemes (CCs) in wireless networks. From all the experiments, we filter out the experiments of downloading a 32M file, and record the throughput of each experiment. In most cases, the download time of a 32M file is longer than 5 seconds, which is enough for CCs to explore the link capacity. The final results are presented in Fig. 2. The packet scheduling scheme for all the experiments is MinRTT, and the primary subflow is established via WLAN.

In this section, we present our insights followed by detailed analysis and discussion.

LIA, OLIA and BALIA are more friendly to single-path TCP, and OLIA shows its superiority in shifting traffic to less congested paths. Overall, the CCs designed especially for MPTCP achieve lower throughput than others because they

slow down the increment of subflows' congestion windows to achieve friendliness with single-path TCP. Moreover, in the Corridor case (Fig. 2(b)), OLIA outperforms the other two. That is because it transmits 63.8% of all traffic to the WLAN subflow (the number for LIA and BALIA is 36.2% and 48.9%, respectively) thanks to its design of preferring subflows with lower RTTs (see Table III). Note that even though the subflows' congestion windows grow more slowly, the overall throughput of MPTCP is still higher than that of TCP because MPTCP can leverage the bandwidth of multiple paths to achieve higher maximum throughput. Besides, the delay-based wVegas algorithm suffers very low throughput that it can hardly be adequate in real networks.

BBR is the most efficient CC for single-path TCP in wireless scenarios, and its ability to increase throughput also benefits MPTCP. For single-path TCP, BBR does not underperform any other CC in all three locations. Especially in the Corridor case (Fig. 2(b)), BBR performs much better than the other schemes when using a public Wi-Fi network which is unstable (with a long error bar) and may suffer packet loss. In this case, BBR's "congestion-based" design can reduce transmission delay and will not decrease its throughput on packet loss. Thus it can achieve higher throughput.

However, *BBR's ignorance of packet loss may cause declined MPTCP performance in some cases.* In the Dorm case (Fig. 2(c)) where the WLAN subflow is built over a private AP with rate control, BBR underperforms both Reno and CUBIC in multipath cases, even though they achieve similar throughput in single-path cases. To understand the reason, we capture the packets of two MPTCP connections with BBR and Reno using tcpdump, and present the number of all transmitted packets and retransmitted packets on the WLAN subflow¹ in Fig. 3. The figure illustrates that BBR causes more retransmissions than Reno on the WLAN subflow. Specifically, when using BBR, the traffic on the WLAN subflow is in a "burst" pattern, which means BBR sends packets at a high rate but is then throttled by the rate control of the AP. That results in many packet losses on the WLAN subflow and further blocks the transmission of the other subflow: Packets from the LTE subflow have to wait for the lost packets on the WLAN subflow for in-order delivery. Before the lost packets are recovered, these out-of-order packets are stored in the receive buffer. The overly occupied receive buffer causes a small MPTCP to send

¹The retransmissions captured on the LTE subflow is negligible.

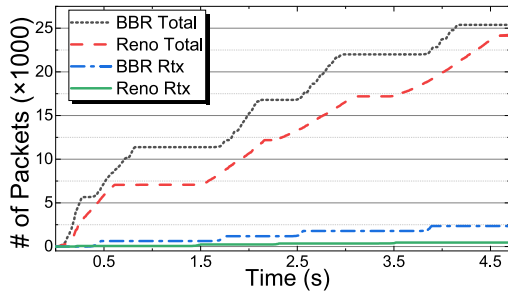


Fig. 3. BBR causes more packet retransmissions on the WLAN subflow than Reno in the Dorm case.

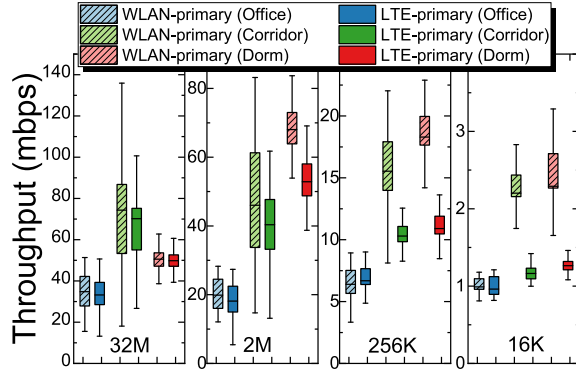


Fig. 4. MPTCP's average throughput with different primary subflow.

window that blocks the transmission on both subflows and also increases the risk of buffer overflow, dragging down the throughput of the whole connection. On the contrary, Reno cuts down its congestion window on packet losses and sends packets more smoothly than BBR. In this way, Reno reduces the number of packet losses and still provides throughput similar to BBR, so that it will not block the transmission of the other subflow like BBR does. In our subsequent measurement of various packet scheduling schemes, we discover that the observed performance decline might be attributed to an inappropriate packet scheduling scheme. Since the design of packet scheduling schemes does not consider the effects of different congestion control schemes, in certain scenarios, although single-path BBR exhibits higher throughput compared to other schemes, multipath BBR underperforms relative to those same packet scheduling schemes.

B. Path Management

This part focuses on the influence of path management schemes, which refer to Fullmesh with different primary subflow selections. We present the throughput of downloading different-sized files in the three locations in Fig. 4. In these experiments, MPTCP uses BBR for congestion control and MinRTT for packet scheduling. In the three locations, the network conditions are very different: In the Office, the two paths both use cellular networks, thus are homogeneous; while in the Corridor and Dorm cases, single-path flows using Wi-Fi networks can reach much higher throughput than those using cellular networks, and thus they are considered heterogeneous.

TABLE IV
THE RATIO OF BYTES SENT ON THE PRIMARY SUBFLOW IN HETEROGENEOUS NETWORK CASES

Size	Corridor		Dorm	
	MP-WLAN	MP-LTE	MP-WLAN	MP-LTE
32M	66.44%	37.35%	67.84%	32.91%
2M	75.35%	25.90%	97.59%	6.87%
256K	97.09%	29.61%	97.10%	27.07%
16K	98.93%	94.46%	98.93%	94.25%

We can observe that *primary subflow selection significantly influences the throughput of small or medium-sized flows*. For small 16K flows, establishing a primary subflow on the WLAN path with lower RTT and higher link capacity determines the maximized throughput in the Corridor and Dorm cases. That is because such small flows finish before the second subflow is established. As shown in Table IV,² the data sent on the primary subflow is nearly 100% of the total data. For medium-sized 256K and 2M flows, the primary subflow selection still significantly influences the overall performance. In LTE-primary cases, before the second WLAN subflow is established, the LTE subflow carries a relatively small fraction of data. Even though the second subflow on the WLAN path carries the majority of data, the overall throughput is still dragged down by the data sent to the LTE subflow. On the contrary, the WLAN-primary MPTCP flows can send most of the data on the primary subflow. Thus, the overall throughput is much higher than LTE-primary ones. Here we observe that in LTE-primary cases, the secondary WLAN subflow carries even a larger fraction of data for the 2M file than the 256K one. That is because the throughput of the WLAN subflow grows much faster and can reach much higher than the LTE one during the short completion time with low RTT and high link capacity. That means the WLAN subflow can send more data before the LTE subflow grows up to its maximum throughput.

Besides, *primary subflow selection does not influence large flows so much*. In this case, the two subflows reach stable throughput and are fully utilized during the long flow completion time, so the difference between the primary and secondary subflow is minimal. Table IV shows that in each location, both WLAN-primary and LTE-primary flows send over 60% of the 32M data on the WLAN subflow, leading to similar throughput as shown in Fig. 4.

C. Packet Scheduling

In this part, we focus on the comparisons among different packet scheduling schemes. From all the experiments, we filter the experiments of downloading files of different sizes (i.e., 32M and 2M) and present the average throughput of different scheduling schemes with the same experimental settings in rows of Table V. To make the data easier to read and compare, the throughput in each row is normalized by the maximal throughput of that row. As for the CC schemes, we choose the congestion-based BBR and the loss-based CUBIC for congestion control.

²MP-WLAN/MP-LTE denotes that the WLAN/LTE subflow is the primary one for the MPTCP connection.

TABLE V
NORMALIZED AVERAGE THROUGHPUT OF DIFFERENT PACKET
SCHEDULING SCHEMES

	CCs	scheduling schemes					Single-path	
		MinRTT	RR	ReMP	ECF	BLEST	WLAN	LTE
Office	BBR	0.93	1.00	0.61	0.96	0.86	0.44	0.62
	CUBIC	0.96	0.98	0.65	0.99	1.00	0.46	0.65
Corridor	BBR	0.91	0.81	0.87	0.84	1.00	0.96	0.35
	CUBIC	0.92	0.89	0.77	0.88	1.00	0.55	0.42
Dorm	BBR	1.00	0.96	0.82	1.00	0.83	0.74	0.44
	CUBIC	0.97	0.93	0.83	1.00	0.95	0.69	0.41
32M	AVG.	0.96	0.93	0.75	0.95	0.94	-	-
Office	BBR	0.96	1.00	0.76	0.93	0.81	0.67	0.79
	CUBIC	0.97	0.99	0.74	1.00	0.93	0.84	0.88
Corridor	BBR	1.00	0.74	0.93	0.82	0.91	0.86	0.45
	CUBIC	0.97	0.93	0.86	0.92	1.00	0.68	0.42
Dorm	BBR	0.70	0.54	0.73	0.70	0.80	1.00	0.19
	CUBIC	0.96	0.94	0.94	1.00	0.92	0.89	0.28
2M	AVG.	0.92	0.86	0.83	0.90	0.90	-	-

MinRTT is the most effective scheduling scheme for general performance. Overall, *MinRTT* achieves the highest average normalized throughput because it fully uses multipath bandwidth. For the same reason, *RR* can also achieve high throughput in the Office cases with homogeneous network conditions. However, *ECF* and *BLEST* may underutilize the bandwidth of a subflow. That is because they will sometimes “wait” for their preferred subflow to become available instead of filling the congestion windows of both subflows. Such a design makes them underperform *MinRTT* in most cases where the network conditions of the two paths are not very different. To sum up, *MinRTT* is a good choice for general MPTCP transmission for its simplicity and effectiveness.

However, in highly heterogeneous network scenarios, the well-designed *ECF* and *BLEST* show their superiority. In this case, *MinRTT* and *RR* underperforms because they result in the out-of-order delivery issue. That is, packets from a fast subflow (a subflow with low RTT and/or high throughput) may have to wait for the packet from a slow subflow (a subflow with high RTT and/or low throughput) to ensure MPTCP’s in-order delivery. Before the stalled packets with lower sequence numbers arrive, these out-of-order packets are stored in the receive buffer and cause head-of-line (HoL) blocking. In other words, the occupied receive buffer with limited size may lead to a small receive window that blocks the transmission of the fast subflow. In the worst case, out-of-order packets may even overflow the buffer and cause dramatic throughput degradation. Such an issue occurs in the Corridor case using BBR and the file size is 32M (Corridor, BBR, 32M). Fig. 5(a) shows the detailed throughput in this heterogeneous scenario, where the average throughput of single-path TCP flow on the WLAN path is 2.77 times that on the LTE path, and the RTT of the WLAN path is around half that of the LTE path. In such a case, *MinRTT* and *RR* result in low normalized throughput, which are 0.91 and 0.81, respectively. On the other hand, *BLEST* achieves the highest throughput by scheduling packets more precisely. Basically, they take more information (e.g., window size, RTT, queue in send buffer, etc.) into consideration and schedule packets in a way that can reduce the HoL blocking caused by heterogeneous paths.

Fig. 5(b) shows another heterogeneous scenario (Dorm, BBR, 2M), where the average throughput of single-path TCP

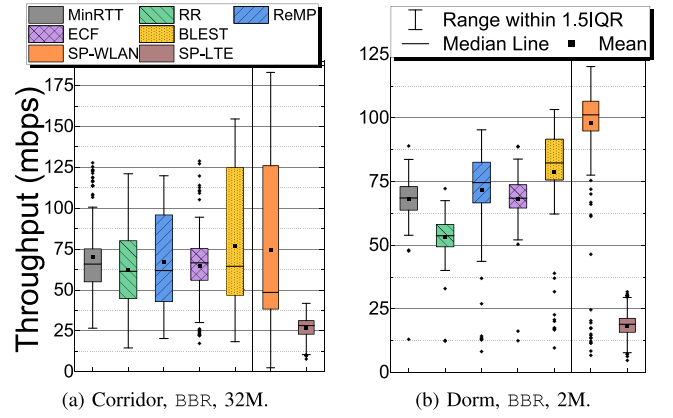


Fig. 5. MPTCP’s average throughput in heterogeneous network scenarios. The figures include detailed information in Table V.

flow on the WLAN path is more than 5 times that on the LTE path. In this case, using the LTE subflow results in declined throughput, and none of the scheduling schemes can achieve higher throughput than using single-path TCP on the WLAN, and *MinRTT* and *RR* perform the worst. That is because these two scheduling schemes send packets to the LTE subflow regardless of its poor conditions, resulting in a long re-ordering delay. When transmitting a medium-sized or small file, the re-ordering delay may take up a major portion of the transmission time. On the contrary, *BLEST* is more cautious about using the LTE subflow and thus achieves higher throughput. Besides, *ReMP* also performs better than *MinRTT* in this case. Though in Table V, *ReMP* seems to be unattractive since it rarely achieves high throughput, we show that it provides an essential scheduling scheme that guarantees MPTCP’s performance in harsh network environments (i.e., highly heterogeneous, with high latency and/or packet loss) in Section V.

D. A Drawback of MPTCP Packet Scheduling

Up until this point, we have primarily highlighted the advantages of MPTCP. The experimental results demonstrate that MPTCP has the ability to aggregate the bandwidth of multiple subflows and achieve higher throughput/goodput in most scenarios. However, during our measurements, we notice that MPTCP sometimes suffers an unexpected decline in throughput but recovers after a period without changing any scheme or experimental setting. MPTCP fails to aggregate bandwidth in these cases and may even provide lower throughput than single-path TCP flows. This phenomenon is caused by inappropriate packet scheduling in harsh³ wireless networks. When the subflows suffer harsh network conditions, scheduling packets to these subflows may cause a series of issues, including re-ordering delay, HoL blocking, and receive buffer overflow, leading to poor transmission performance of MPTCP.

The occasional poor performance of MPTCP is non-negligible, and we can catch a glimpse of their negative effects on the overall performance in Fig. 5(a). This figure illustrates

³In this paper, we use “harsh” conditions to describe network conditions with high delay, low throughput, and/or packet loss.

that the throughput of MPTCP varies a lot with a subflow on an unstable WLAN path using a public AP. The minimum of the whiskers and lower quartile of the boxes show that in quite many cases, MPTCP suffers a significant decline in throughput due to the harsh network conditions.

When deploying MPTCP in practical scenarios, it is essential to provide quality-guaranteed transmission in various scenarios, including harsh ones. Although the packet scheduling schemes mentioned above can be effective in their targeted scenarios, there is not a particular one that can always provide satisfactory performance under different scenarios. That means if an MPTCP server fixes its scheduling scheme, it has to risk suffering poor performance when the scheduling scheme does not suit the current network scenario. This shortcoming undermines the effectiveness of MPTCP. Based on the performance measurement of various combinations of different MPTCP components, we recognize that the interaction effects between congestion control and packet scheduling schemes cannot be overlooked. Although they can be designed and implemented independently, different congestion control schemes result in varied performances among packet scheduling schemes. This variability makes it challenging to design a model-based heuristic packet scheduling scheme that adapts to all congestion control schemes.

In Section V, we first introduce an adaptive framework to integrate the advantages of different schemes, and then focus on the packet scheduling task in MPTCP. We propose OLSch to adaptively select the best-performing scheduling scheme, based on its effectiveness, thus operating independently of the constraints imposed by any specific congestion control scheme. We conduct additional experiments⁴ in a testbed as well as in real networks to study the performance of MPTCP in harsh network environments.

V. ADAPTIVE FRAMEWORK TO INTEGRATE MPTCP SCHEMES

A. Overview

In Section IV-D, we discuss the drawback of existing in-kernel MPTCP schemes that a single heuristic scheme cannot perform well in all scenarios. These schemes perform optimally in their respective scenarios, but in certain situations, they perform poorly. Designing new schemes is time-consuming and requires prolonged testing to validate their stability in the real world. Nevertheless, existing schemes offer sufficient inspiration. If each MPTCP connection can choose the scheme that performs the best under current network conditions, its performance should be significantly improved. Based on this idea, we propose an adaptive framework to integrate MPTCP schemes (named OLSch). As sketched in Fig. 6, OLSch is an extensible framework and can be used for integrating all available schemes in MPTCP kernel. The core workflow is that OLSch selects a set of schemes to form the scheme pool for scheme selection, and then learns and exploits the optimal one to generate strategy according to the current transmission conditions logs

⁴The experiments with declined performance in what we present above are conducted at different times, under distinct network conditions, and not mutually comparable. Thus, comparisons among them are not included here.

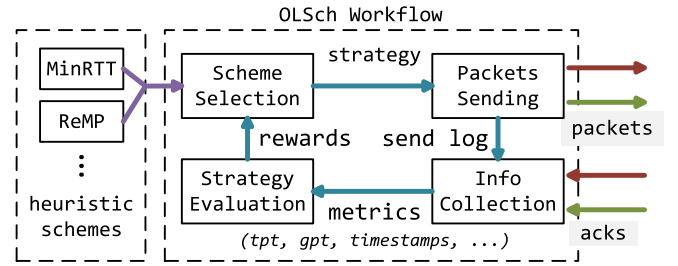


Fig. 6. The framework of OLSch.

and network feedback. In this way, OLSch can harness the strengths of existing schemes, complementing each other's weaknesses, and maintain high performance across complex wireless networks.

Particularly, OLSch provides an online-learning-driven approach to integrate multiple schemes. Then it can learn and exploit the optimal one according to current network conditions to achieve adaptive scheme selection. We formulate the packet scheduling task as a multi-armed bandit (MAB) problem. We make the following efforts to turn the generic online-learning algorithm into a practical scheme selection: **1)** We design a reward function to quantify the transmission performance of each scheme so that OLSch can evaluate the schemes by their rewards. **2)** We design the workflow for OLSch, which includes information collection, scheme evaluation, and scheme selection. It defines the specific process of how OLSch works. **3)** Given the OLSch workflow, OLSch provides a framework to work with any existing schemes.

In our implementation, OLSch integrates MinRTT and ReMP into the candidate scheme pool. The synergy between MinRTT and ReMP offers a balanced and efficient solution, supported by the test results. MinRTT, despite its simplicity, excels in most scenarios. Additionally, in instances where MinRTT underperforms, ReMP effectively mitigates its shortcomings, making these schemes highly complementary. Conversely, ECF and BLEST are not included in the pool. ECF is not included due to its inferior performance, especially in environments with small receive buffers, where it significantly underperforms compared to MinRTT. BLEST, reliant on feedback from previous scheduling decisions for adjustments, poses integration challenges when used with other schemes in our framework. This susceptibility to errors in adjustments makes BLEST less suitable for inclusion without a more sophisticated integration strategy.

Generally, OLSch learns an efficient scheduling scheme that provides the highest throughput for each specific MPTCP connection under different conditions. As an online-learning-driven method, it is deployable and can run without any prior knowledge or offline training. Further experimental evaluations in our testbed and real wireless network show that it provides consistently good performance in different scenarios.

B. Design and Implementation of OLSch

Here, we explain how to map the scheme selection to an MAB problem. To begin with, we present a description of the MAB problem.

In a K -armed bandit problem, a gambler must choose an arm of K slot machines to play in a sequence of trials. Playing machine i in trials $(t_1, t_2, \dots)$ yield rewards $(x_i(t_1), x_i(t_2), \dots)$, which are independent and identically distributed according to an unknown law with unknown expectation. The gambler's goal is to maximize the summarized reward. To achieve this goal, the gambler must find a balance between exploiting the arm with the highest reward currently and exploring the others to find a possibly better arm.

Given the description, the packet scheduling problem can perfectly suit the framework of the MAB problem. The scheduling scheme can be regarded as a gambler that chooses one of the K given schemes for a period. During the period, OLSch sends packets following the strategy generated by the chosen scheme and obtains a reward for the scheme at the end of the period with a reward function. A well-designed reward function reflects the performance of the chosen scheme, so the goal of achieving better transmission performance is transformed into maximizing the summarized reward. Then, OLSch starts a new period by choosing a scheme with a certain policy. The policy is the key to balancing exploitation and exploration for maximized rewards. OLSch leverages UCB1 algorithm [43] as the policy.

UCB1 [43] is an effective algorithm to solve the MAB problem. Basically, UCB1 algorithm maximizes the summarized reward by choosing an arm that maximizes the sum of two terms (line 6 in Algorithm 1). The first term is the current average reward of the scheme. The second term is related to the size (according to Chernoff-Hoeffding bounds) of the one-sided confidence interval for the average reward within which the true expected reward falls with overwhelming probability. The UCB1 algorithm guarantees that the cumulative regret grows logarithmically with time, which is asymptotically optimal for this type of problem. Based on the UCB1 algorithm, OLSch can balance the exploration of different scheduling schemes and exploit the one that performs best based on previous decisions. Still, we need to carefully design the reward function, the workflow, and the actions to turn the generic MAB algorithm into OLSch.

Reward Function: Reward function is the key for scheme evaluation in Fig. 6. It quantifies the transmission performance of an MPTCP connection in a period and yields a high or low reward to reflect whether the chosen scheme performs well in the period or not. We define the reward of the chosen scheme s in period t as:

$$x_s(t) = \frac{gpt(t)}{\sum_{i=1}^N \max_{\tau \in [t-T, t]} tpt_i(\tau)} \in [0, 1], \quad (1)$$

where $gpt(t)$ denotes the goodput of the MPTCP connection in period t , $tpt_i(t)$ denotes the throughput of subflow i in period t , and N is the number of subflows.

Though achieving high goodput⁵ is the main objective, the absolute value of the goodput is not a good choice as a reward. That is because it can vary a lot due to the random fluctuations in wireless networks, and such fluctuations do not

Algorithm 1: OLSch Workflow

Parameters: s , the chosen scheme.
 $x_s \in [0, 1]$, current reward of s .
 $\bar{x}_s \in [0, 1]$, average reward of s .
 n_s , the number of times to choose s .
 n , the overall number of trials.
 i , the subflow index.

```

1 Function SchemeSelection():
2   ▷ Run until the connection is closed.
3   if a scheme  $s$  has never been chosen then
4     | Choose  $s$ .                                ▷ Initialization
5   else
6     | Choose  $s$  that maximizes  $\bar{x}_s + \sqrt{\frac{2 \ln n}{n_s}}$ .  ▷ UCB1
7   Send packets following  $s$  in the next period.
8   while during the period do
9     | Collect information from packets and ACKs.
10  ▷ At the end of the period
11  Get throughput ( $tpt_i$ ) of each subflow  $i$ . ▷ Eq. (2)
12  Get goodput ( $gpt$ ) of the connection.    ▷ Eq. (2)
13  Get reward  $x_s$ .                          ▷ Eq. (1)
14  Update the parameters of action  $s$ .

```

reflect the performance of the chosen scheme. Instead, Eq. (1), which represents the efficiency of the scheme s to utilize the subflows, can mitigate the effects of random fluctuations. For example, if the throughput of a subflow decreases due to some network issues, the absolute value of the goodput will also decrease, but it does not mean the scheme performs poorly. In this case, Eq. (1) will still yield high rewards if the scheme does well in aggregating bandwidths and finally achieves a goodput close to the sum of throughput of all subflows.

In the denominator of Eq. (1), we use the maximum throughput of each subflow in the last T periods⁶ ($\max_{\tau \in [t-T, t]} tpt_i(\tau)$) instead of the throughput in the current period ($tpt_i(t)$). That is because a suboptimal scheme may cause a declined throughput of subflows as well as a low goodput, which means $tpt_i(t)$ and $gpt(t)$ can both be small. In this case, $\frac{gpt(t)}{\sum_{i=1}^N tpt_i(t)}$ will yield an unwanted high reward, but our reward function will not. To sum up, using the maximum throughput correctly reflects the performance of the chosen scheme.

Workflow: We design a workflow that defines the working process of OLSch which is described in Algorithm 1. OLSch works for the whole lifetime of an MPTCP connection. It divides time into *decision periods*. In each period, OLSch will choose one of the given schemes to generate strategies and send packets accordingly.

In our implementation, OLSch integrates different scheduling schemes. First, we specify the K scheduling schemes that OLSch can select. The choice of the schemes will be discussed later. Then, OLSch integrated with K scheduling schemes works as follows: In the beginning, OLSch starts the first

⁵In this section, the term “goodput” denotes the throughput of an MPTCP connection to distinguish itself from the throughput of a subflow.

⁶In our implementation, T is the number of the periods in the last 4 seconds.

K decision periods by sequentially choosing one of the K given schemes. During each period (2.5 times the maximal RTT of all subflows in our implementation), OLSch behaves the same as the chosen scheme. When sending packets, the connection collects information for reward calculation, such as the bytes of sent data at subflow i (b_i) and at connection level (B_c), the timestamps of the first sent packet (t_b) and the last acknowledgment (t_e) of the period, etc. At the end of the decision period, OLSch calculates the goodput of the connection (gpt) as well as the throughput of subflow i (tpt_i). Specifically, we have:

$$gpt = \frac{B_c}{t_e - t_b} \quad \text{and} \quad tpt_i = \frac{b_i}{t_e - t_b}. \quad (2)$$

Here, we omit the parameter t because all the calculations here are in the same period. OLSch evaluates the performance of the scheme by calculating its reward with Eq. (1), and it updates the parameters as shown in Algorithm 1. Finally, it ends the current decision period and starts a new one. Except for the first K periods, OLSch will choose the scheme that maximizes the sum of the average reward and the upper bound (line 6 in Algorithm 1) to chase the maximized rewards.

Actions: With the reward function and workflow, OLSch can now evaluate each of the given schemes and select one of them sequentially for maximized rewards. To finish the design of OLSch, we need to choose K schemes as its actions.

The choices of the scheduling schemes are based on our measurements. In Section IV, the experimental results illustrate that, on average, MinRTT is the most effective one to provide high throughput. But we also find that it sometimes causes declined throughput when the subflows are suffering high delay, packet loss, or heterogeneous network paths. On the contrary, ReMP provides stable performance in such harsh network scenarios but fails to achieve bandwidth aggregation. These two schemes work well in a complementary way. Therefore, OLSch adopts them as its actions and uses one of them in the right scenarios to improve the overall transmission performance, which makes $K = 2$.

OLSch is an online scheme based on UCB1 [43] to choose scheduling strategy. The computational complexity of a single round of UCB1 is $O(K)$, and the total complexity across all rounds is $O(KN)$, where K is the number of arms and N is the number of iterations. Since OLSch only updates and makes decisions based on RTT cycles, the computational complexity for each RTT cycle is $O(K)$. As K is small in practice, the decision logic incurs negligible overhead during transmission, making OLSch well suited for practical deployment.

Specifically, selecting MinRTT means that during the period, OLSch will send normal data on all subflows. Opting for ReMP means that during the same period, redundant copies of packets will be sent on all subflows except for the one with the lowest RTT. Consequently, OLSch can adeptly switch between these two straightforward schemes at appropriate times, effectively aggregating bandwidth while reducing out-of-order packets and HoL blocking. Although BLEST is not included in the alternate scheme pool due to integration challenges within the scheduling framework, this implementation still manages to achieve the objectives envisioned by

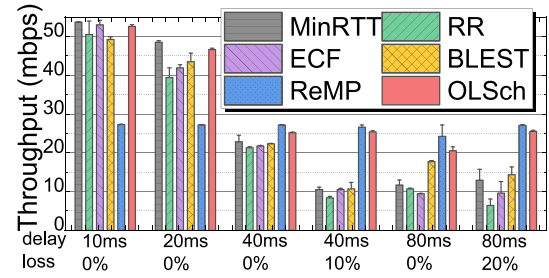


Fig. 7. The average throughput of different scheduling schemes.

BLEST, yielding satisfactory performance. However, we have identified certain limitations with OLSch. Specifically, it only selects heuristic scheduling schemes and makes decisions periodically rather than per-packet to balance complexity and usability. This method is not flexible enough to fully capitalize on existing schemes. For enhanced performance, a future approach could involve action imitation rather than simply selecting schemes. This would involve emulating the output of the most effective scheme, thereby circumventing integration conflicts among different schemes and leveraging the strengths of a broader array of schemes for more beneficial results.

C. Performance Evaluation

In this section, we implement OLSch as a Linux kernel module and evaluate it together with other in-kernel scheduling schemes in our testbed and the real networks.

1) Evaluation in Testbed: Since the parameters in real networks, e.g., link capacity, delay, packet loss rate, and so on, cannot be controlled easily, we first evaluate OLSch in a testbed. In the testbed, we connect two real hosts with two network paths simulated with NetEm [44]. By tuning the parameters of the simulated network paths (e.g., link capacity, delay, and packet loss rate), we can evaluate OLSch thoroughly over a wide range of network scenarios. For each experiment, a host generates a 100-second MPTCP connection and transmits data with different scheduling schemes, including MinRTT, BLEST, RR, ReMP and our OLSch. The experiments are repeated eight times with each network setting, and the average throughput is presented in Fig. 7. In the experiments, the network settings of path 1 is fixed, which is 32 Mbps bandwidth and 10 ms RTT without random packet loss. The bandwidth of path 2 is 32 Mbps, and we change the settings of path 2 to simulate different network scenarios. The RTT and random packet loss rate of path 2 are also presented in the x-axis of Fig. 7. The receive buffer size of a connection is 256 KB.

The figure illustrates that OLSch performs well in all the scenarios. Specially, when the RTT of path 2 is 10 ms and 20 ms, OLSch, MinRTT, BLEST and RR achieve much higher throughput than ReMP. In these cases, they can fully utilize the network resources of the two paths. Thus, all of them can achieve high throughput. However, since ReMP sends packets redundantly to all subflows, it can only achieve half of the highest throughput.

On the other hand, when the subflows are heterogeneous or suffer random packet loss, MinRTT, BLEST, and RR cause

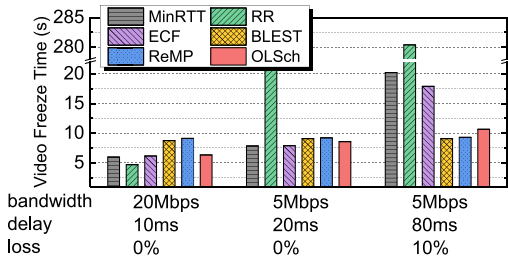


Fig. 8. The performance of video-on-demand.

a dramatic decline in throughput. In such cases, packets sent to different subflows will arrive out of order and cause HoL blocking. With a limited buffer size, the blocking further limits the receive window size and thus drags down the throughput of the other subflow. Also, random packet loss even exacerbates such issues because it takes a long time to retransmit lost packets. ReMP is much more suitable in such cases: the delayed and lost packets on path 2 are also redundantly sent to path 1, so they have no impact on the transmission performance. As a result, ReMP can fully utilize path 1 without the negative effects from path 2, and thus it achieves higher throughput than other scheduling schemes. Our OLSch learns that ReMP's scheme is optimal so that it performs well in these cases too. To sum up, in all the cases in our experiments, OLSch can always schedule packets with the optimal scheme that performs the best under the current network scenario.

To demonstrate the flexibility of OLSch across a broader range of applications, we evaluate the performance in video-on-demand (VoD) and real-time video streaming applications. In the experiments, path 1 is configured with 20Mbps bandwidth, 5 ms delay, and 0% packet loss rate. We vary the settings of path 2 to simulate different network conditions. Fig. 8 shows the performance of VoD. For each VoD, the video is split into several segments for transmission, with each segment containing a few seconds of video. The MPTCP receive buffer size is set at 256 KB. Overall, OLSch exhibits robust performance across all scenarios. When both paths are in optimal condition, MinRTT achieves nearly the best results, with OLSch maintaining this high performance by adaptively selecting MinRTT. Conversely, in scenarios with significant heterogeneity between the two paths, ReMP delivers superior performance, and OLSch effectively adapts to match ReMP's superior performance.

Fig. 9 illustrates the performance of average frames per second (FPS) of real-time video streaming. In each experiment, a sender generates a 100-second real-time video streaming and sequentially transmits the video frames to the receiver. OLSch achieves sub-optimal performance for real-time video streaming. Since real-time video streaming requires the sequential transmission of video frames, where any blocking of individual frames can significantly impair its performance. BLEST, which is designed to predict and schedule packets to avoid such blocking, is highly suitable for real-time video streaming. Nevertheless, OLSch remains

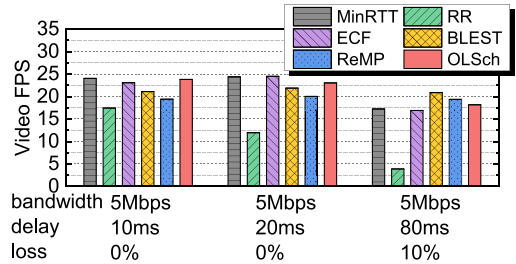


Fig. 9. The performance of real-time video streaming.

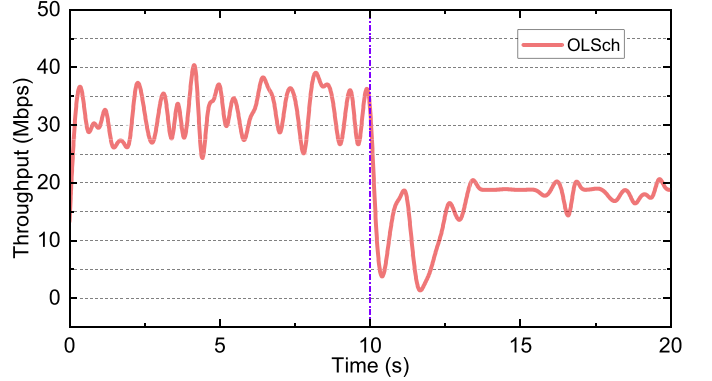


Fig. 10. The convergence of OLSch.

robust across various path conditions. Nevertheless, OLSch still remains robust across various path conditions.

We also test the convergence performance of OLSch when network conditions change. In this experiment, we initiate an MPTCP connection with both paths initially configured at 20 Mbps, 10 ms delay, and 0% packet loss. From the 10th second onward, the delay on one path increases to 80 ms, and the packet loss rises to 10%. Fig. 10 illustrates the real-time throughput of the MPTCP connection. The results demonstrate that OLSch quickly converges to the new state approximately 3 seconds after the network conditions change.

Note that OLSch does not always achieve the best performance due to the online learning process. The online learning process requires no prior knowledge of strategy performance. However, it necessitates an initial exploration phase, during which several RTT cycles are required to converge to the optimal strategy. Similarly, upon changes in network conditions, OLSch incurs a re-exploration period, taking additional cycles to adapt and converge to a new optimal strategy. However, considering both the integrated performance and computational overhead, OLSch's ability to adapt to different network conditions still makes it the best choice from the perspective of a comprehensive evaluation.

2) *Evaluation in Wireless Networks:* We also evaluate OLSch in real networks with our measurement platform. To make comparisons among the schemes in harsh networks (e.g., with heterogeneous conditions, high delay, and packet loss), we use NetEm to introduce random delay and packet loss to one of the network paths to provide more controllable

TABLE VI
THE AVERAGE NORMALIZED THROUGHPUT OF
DIFFERENT SCHEDULING SCHEMES

Dly. & Loss	# of Grp.	MinRTT	ReMP	OLSch	RR	ECF	BLEST
w/	52	0.72	0.93	0.90	0.76	0.73	0.82
w/o	54	0.95	0.78	0.91	0.95	0.95	0.88
overall	108	0.84	0.86	0.91	0.86	0.84	0.85

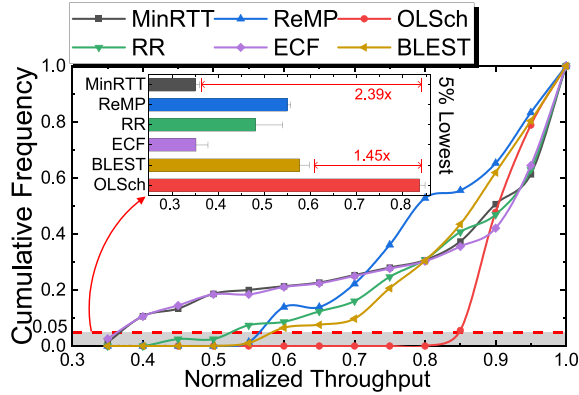


Fig. 11. CDF of the overall normalized throughput. OLSch eliminates the dramatic throughput degradation of other schedules.

environments for the experiments. For each of our three clients, we randomly introduce extra delay and packet loss rate to one of the network paths. The delay (or the packet loss rate) is uniformly distributed in the range of 0 ms to 300 ms (or 0% to 30%). The experiments of the same setting last for 40 minutes, within which the experiments of different scheduling schemes are considered to be in a group and comparable. We also conduct experiments without extra delay and packet loss for comparison. For each experiment, the client requests a 100M file, and the throughput is measured. All results are arranged in chronological order. Since the results vary a lot due to the random network conditions, we divide the average throughput of the scheduling schemes by the maximum in the group for ease of comparison.

First, we present the normalized average throughput in Table VI. The results show that w/ or w/o extra delay and loss, OLSch always achieves high normalized throughput over 0.9, while other scheduling schemes can only reach 0.8 or lower in one of the two cases. Compared to its integrated scheduling schemes, OLSch increases the average throughput by 24.8% compared to MinRTT with extra delay and packet loss; and it increases the average throughput by 16.4% compared to ReMP without extra delay and packet loss.

Fig. 11 shows OLSch's superiority in eliminating the dramatic throughput degradation. OLSch does not suffer from low normalized throughput under 0.8, while over 30% of the experiments of other scheduling schemes suffer normalized throughput lower than that. Particularly, OLSch makes a marked improvement in the "95th worst-case throughput", which refers to the normalized throughput under the cumulative frequency of 0.05 in Fig. 11. Existing in-kernel scheduling schemes only attain 0.58 or lower throughput for the 95th

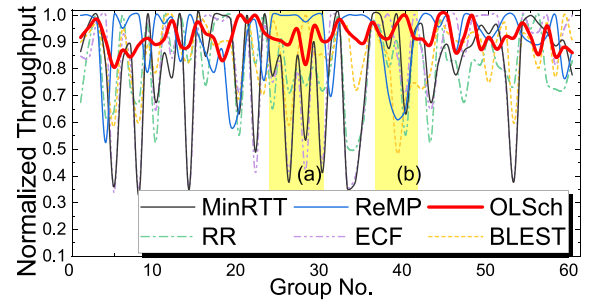


Fig. 12. The normalized throughput of 60 groups in wireless networks.

worst-case, while OLSch can achieve 0.84, which is a substantial improvement, ranging from 1.45 times to 2.39 times better than the other scheduling schemes.

We further present the normalized throughput of all scheduling schemes in 60 consecutive groups in Fig. 12, which shows OLSch's ability to adapt to sophisticated wireless scenarios. When in harsh network conditions where MinRTT and other scheduling schemes suffer declined throughput, OLSch learns the optimal scheme and behaves like ReMP (for example, the case a in Fig. 12); when in good network conditions, OLSch behaves like MinRTT to aggregate bandwidth for higher throughput (the case (b) in Fig. 12).

To sum up, OLSch is the best choice for MPTCP in sophisticated and ever-changing wireless networks from the perspective of a comprehensive evaluation. It does particularly well in eliminating the dramatic throughput degradation caused by harsh network conditions.

VI. CONCLUSION

We presented an in-depth measurement study on MPTCP in wireless networks, featuring comparisons among different schemes for congestion control, packet scheduling, and path management. This paper sheds light on the appropriate scenarios for using these schemes and identifies situations where they could lead to a decline in performance. These insights can help people better understand the performance of MPTCP with different schemes and use them properly when deploying MPTCP. We further proposed a framework named OLSch to integrate different schemes and guarantee MPTCP's performance in various network scenarios. OLSch is implemented in Linux kernel with selected scheduling schemes. Experimental evaluations show that OLSch greatly improves the worst-case performance compared to any other in-kernel scheduling schemes and provides quality-guaranteed performance in all cases.

REFERENCES

- [1] "Cisco annual Internet report (2018–2023) white paper." 2020. Accessed: Jun. 2025. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>
- [2] Y. Li et al., "A nationwide study on cellular reliability: Measurement, analysis, and enhancements," in *Proc. Conf. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2021, pp. 597–609.
- [3] J. R. Iyengar, P. D. Amer, and R. Stewart, "Concurrent multipath transfer using SCTP multihoming over independent end-to-end paths," *IEEE/ACM Trans. Netw.*, vol. 14, no. 5, pp. 951–964, Oct. 2006.

- [4] Q. De Coninck and O. Bonaventure, "Multipath QUIC: Design and evaluation," in *Proc. 13th Int. Conf. Emerg. Netw. Exp. Technol. (CoNEXT)*, 2017, pp. 160–166.
- [5] A. Ford, C. Raiciu, M. Handley, and O. Bonaventure, "TCP extensions for multipath operation with multiple addresses," Internet Eng. Task Force, RFC 8684, 2020. Accessed: Jun. 2025. [Online]. Available: <https://www.ietf.org/rfc/rfc8684.txt>
- [6] Y. Lu et al., "Multi-path transport for RDMA in Datacenters," in *Proc. 15th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2018, pp. 357–371.
- [7] C. Paasch and O. Bonaventure, "Multipath TCP," *Commun. ACM*, vol. 57, no. 4, pp. 51–57, 2014.
- [8] C. Raiciu, S. Barre, C. Pluntke, A. Greenhalgh, D. Wischik, and M. Handley, "Improving Datacenter performance and robustness with multipath TCP," in *Proc. Conf. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2011, pp. 266–277.
- [9] L. Li et al., "A measurement study on multi-path TCP with multiple cellular carriers on high speed rails," in *Proc. Conf. ACM Special Interest Group Data Commun. (SIGCOMM)*, 2018, pp. 161–175.
- [10] O. Bonaventure, "Apple music on iOS13 uses multipath TCP through load-balancers." 2019. Accessed: Jun. 2025. [Online]. Available: http://blog.multipath-tcp.org/blog/html/2019/10/27/apple_music_on_ios13_uses_multipath_tcp_through_load_balancers.html
- [11] "Use multipath TCP to create backup connections for iOS." Apple. 2017. Accessed: Jun. 2025. [Online]. Available: <https://support.apple.com/en-us/HT201373>
- [12] O. Bonaventure and S. Seo, "Multipath TCP deployments," *IETF J.*, vol. 12, no. 2, pp. 24–27, 2016.
- [13] M. Baerts, "Linux MPTCP upstream project." 2022. Accessed: Jun. 2025. [Online]. Available: https://github.com/multipath-tcp/mptcp_net-next/wiki/
- [14] *System Architecture for the 5G System (5GS); Stage 2 (Release 16), Version 16.7.0*, 3GPP Standard TS 23.501, Dec. 2020. Accessed: Jun. 2025. [Online]. Available: <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144>
- [15] O. Bonaventure, "Recommended multipath TCP configuration." 2014. Accessed: Jun. 2025. [Online]. Available: http://blog.multipath-tcp.org/blog/html/2014/09/16/recommended_multipath_tcp_configuration.html
- [16] C. Raiciu et al., "How hard can it be? Designing and implementing a Deployable Multipath TCP," in *Proc. 9th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2012, pp. 399–412.
- [17] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Commun. ACM*, vol. 60, no. 2, pp. 58–66, 2017.
- [18] J. Han et al., "Leveraging coupled BBR and adaptive packet scheduling to boost MPTCP," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7555–7567, Nov. 2021.
- [19] D. Wischik, C. Raiciu, A. Greenhalgh, and M. Handley, "Design, implementation and evaluation of congestion control for multipath TCP," in *Proc. 8th USENIX Conf. Netw. Syst. Design Implement. (NSDI)*, 2011, pp. 99–112.
- [20] R. Khalili, N. Gast, M. Popovic, and J.-Y. Le Boudec, "MPTCP is not Pareto-optimal: Performance issues and a possible solution," *IEEE/ACM Trans. Netw.*, vol. 21, no. 5, pp. 1651–1665, Oct. 2013.
- [21] A. Frommgen, T. Erbschäuer, A. Buchmann, T. Zimmermann, and K. Wehrle, "ReMP TCP: Low latency multipath TCP," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2016, pp. 1–7.
- [22] S. Ferlin, Ö. Alay, O. Mehani, and R. Boreli, "BLEST: Blocking estimation-based MPTCP scheduler for heterogeneous networks," in *Proc. IFIP Netw. Conf. (IFIP Netw.) Workshops*, 2016, pp. 431–439.
- [23] Y.-S. Lim, E. M. Nahum, D. Towsley, and R. J. Gibbens, "ECF: An MPTCP path scheduler to manage heterogeneous paths," in *Proc. 13th Int. Conf. Emerg. Netw. Exp. Technol. (CoNEXT)*, 2017, pp. 147–159.
- [24] Y. Thomas, M. Karaliopoulos, G. Xylomenos, and G. C. Polyzos, "Low latency friendliness for multipath TCP," *IEEE/ACM Trans. Netw.*, vol. 28, no. 1, pp. 248–261, Feb. 2020.
- [25] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.
- [26] C. Raiciu, M. J. Handley, and D. Wischik, "Coupled congestion control for Multipath transport protocols," Internet Eng. Task Force, RFC 6356, 2011. Accessed: Jun. 2025. [Online]. Available: <https://www.ietf.org/rfc/rfc6356.txt>
- [27] Q. Peng, A. Walid, J. Hwang, and S. H. Low, "Multipath TCP: Analysis, design, and implementation," *IEEE/ACM Trans. Netw.*, vol. 24, no. 1, pp. 596–609, Feb. 2016.
- [28] A. Nikraves, Y. Guo, F. Qian, Z. M. Mao, and S. Sen, "An in-depth understanding of Multipath TCP on mobile devices: Measurement and system design," in *Proc. 22nd Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2016, pp. 189–201.
- [29] C. Paasch and S. Barre, "Multipath TCP in the Linux kernel." 2022. Accessed: Jun. 2025. [Online]. Available: <http://www.multipath-tcp.org/>
- [30] Y. Xing et al., "A low-latency MPTCP scheduler for live video streaming in mobile networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7230–7242, Nov. 2021.
- [31] K. Xue et al., "DPSAF: Forward prediction based dynamic packet scheduling and adjusting with feedback for Multipath TCP in lossy heterogeneous networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 2, pp. 1521–1534, Feb. 2018.
- [32] P. Hurtig, K. Grinnemo, A. Brunstrom, S. Ferlin, O. Alay, and N. Kuhn, "Low-latency scheduling in MPTCP," *IEEE/ACM Trans. Netw.*, vol. 27, no. 1, pp. 302–315, Feb. 2019.
- [33] G. Sarwar, R. Boreli, E. Lochin, A. Mifdaoui, and G. Smith, "Mitigating receiver's buffer blocking by delay aware packet scheduling in multipath data transfer," in *Proc. 27th Int. Conf. Adv. Inf. Netw. Appl. Workshops (WAINA)*, 2013, pp. 1119–1124.
- [34] H. Zhang, W. Li, S. Gao, X. Wang, and B. Ye, "ReLeS: A neural adaptive multipath scheduler based on deep reinforcement learning," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, 2019, pp. 1648–1656.
- [35] J. Han et al., "EdAR: An experience-driven multipath scheduler for seamless handoff in mobile networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6839–6852, Oct. 2023.
- [36] H. Wu, Ö. Alay, A. Brunstrom, S. Ferlin, and G. Caso, "Peekaboo: Learning-based Multipath scheduling for dynamic heterogeneous environments," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2295–2310, Oct. 2020.
- [37] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to Personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 661–670.
- [38] T. Lu, D. Pal, and M. Pal, "Contextual multi-armed bandits," in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, vol. 9, 2010, pp. 485–492.
- [39] C. Paasch, G. Detal, F. Duchene, C. Raiciu, and O. Bonaventure, "Exploring mobile/WiFi handover with multipath TCP," in *Proc. ACM SIGCOMM Workshop Cellular Netw., Oper., Chall., Future Design (CellNet)*, 2012, pp. 31–36.
- [40] Y.-C. Chen, Y.-S. Lim, R. J. Gibbens, E. M. Nahum, R. Khalili, and D. Towsley, "A measurement-based study of multipath TCP performance over wireless networks," in *Proc. Conf. Internet Meas. Conf. (IMC)*, 2013, pp. 455–468.
- [41] Q. De Coninck, M. Baerts, B. Hesmans, and O. Bonaventure, "A first analysis of multipath TCP on smartphones," in *Proc. Passive Act. Meas. (PAM)*, 2016, pp. 57–69.
- [42] Y. Cao, M. Xu, and X. Fu, "Delay-based congestion control for multipath TCP," in *Proc. 20th IEEE Int. Conf. Netw. Protoc. (ICNP)*, 2012, pp. 1–10.
- [43] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, vol. 47, no. 2, pp. 235–256, 2002.
- [44] M. Kerrisk, "Tc-netem(8)—Linux manual page." 2011. Accessed: Jun. 2025. [Online]. Available: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>



Jiangping Han (Member, IEEE) received the B.S. and Ph.D. degrees from the Department of Electronic Engineering and Information Science, USTC in 2016 and 2021, respectively. From November 2019 to October 2021, She was a Visiting Scholar with the School of Computing, Informatics, and Decision Systems Engineering, Arizona State University. She is currently a Postdoctoral researcher with the School of Cyber Science and Technology, USTC. Her research interests include future Internet architecture design and transmission optimization.



Yitao Xing received the B.S. degree in information security from the School of the Gifted Young, University of Science and Technology of China (USTC) in 2018, and the Ph.D. degree in information security from the School of Cyber Science and Technology, USTC in 2023. His research interests include future Internet architecture and transmission optimization.



Jinhao Liu received the B.S. degree in information security from the School of Cyber Science and Technology, University of Science and Technology of China (USTC) in 2023, where he is currently pursuing the Ph.D. degree. His research interests include future Internet architecture and transmission optimization.



Kaiping Xue (Senior Member, IEEE) received the bachelor's degree from the Department of Information Security, University of Science and Technology of China (USTC) in 2003, and the Ph.D. degree from the Department of Electronic Engineering and Information Science (EEIS), USTC in 2007. From May 2012 to May 2013, he was a Postdoctoral Researcher with the Department of Electrical and Computer Engineering, University of Florida. He is currently a Professor with the School of Cyber Science and Technology, USTC, where he

is also the Director of the Network and Information Center. His research interests include next-generation Internet architecture design, transmission optimization, and network security. His work won Best Paper Awards in IEEE MSN 2017 and IEEE HotICN 2019, the Best Paper Honorable Mention in ACM CCS 2022, the Best Paper Runner-Up Award in IEEE MASS 2018, and the Best Track Paper in MSN 2020. He serves on the editorial board of several journals, including the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, the IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and the IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. He has served as a guest editor for many reputed journals/magazines, and he has also served as a track/symposium chair for some reputed conferences. He is an IET Fellow.



Jian Li (Senior Member, IEEE) received the bachelor's degree from the Department of Electronics and Information Engineering, Anhui University in 2015, and the Ph.D. degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC) in 2020. From November 2019 to November 2020, he was a Visiting Scholar with the Department of Electronic and Computer Engineering, University of Florida. From December 2020 to October 2022, he was a Postdoctoral Researcher with the School

of Cyber Science and Technology, USTC, where he is currently an Associate Researcher. His research interests include quantum networking, wireless networks, and network security. He serves as an Editor of *China Communications* and has served as Technical Program Committee Co-Chair for IEEE IWCMC 2025 QNC Symposium.



David S. L. Wei (Life Senior Member, IEEE) received the Ph.D. degree in computer and information science from the University of Pennsylvania in 1991. From May 1993 to August 1997, he was on the Faculty of Computer Science and Engineering, University of Aizu, Japan, (as an Associate Professor and then a Professor). He is currently a Professor with the Computer and Information Science Department, Fordham University. He has authored and co-authored more than 140 technical papers in various archival journals

and conference proceedings. He currently focuses his research efforts on cloud and edge computing, cybersecurity, and quantum computing and communications. He is the recipient of IEEE Region 1 Technological Innovation Award (Academic) in 2020, for contributions to information security in wireless and satellite communications and cyber-physical systems. He was a Lead Guest Editor or a Guest Editor for several special issues in the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, the IEEE TRANSACTIONS ON CLOUD COMPUTING, and the IEEE TRANSACTIONS ON BIG DATA. He also served as an Associate Editor for IEEE TRANSACTIONS ON CLOUD COMPUTING from 2014 to 2018 and for *Journal of Circuits, Systems and Computers* from 2013 to 2018, and an Editor for IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS for the Series on Network Softwarization & Enablers from 2018 to 2020. He is a member of ACM and AAAS, and is a Life Senior Member of IEEE Computer Society and IEEE Communications Society.



Qibin Sun (Fellow, IEEE) received the Ph.D. degree from the Department of Electronic Engineering and Information Science, University of Science and Technology of China in 1997, where he is currently a Professor with the School of Cyber Science and Technology. He has published more than 120 papers in international journals and conferences. His research interests include multimedia security, network intelligence, and security.



Jun Lu received the bachelor's degree from Southeast University in 1985, and the master's degree from the Department of Electronic Engineering and Information Science (EEIS), University of Science and Technology of China in 1988, where he is currently a Professor with the School of Cyber Science and Technology. He is also the President of Jiaxing University. His research interests include theoretical research and system development in the field of integrated electronic information systems, network, and information security. He is an Academician of the Chinese Academy of Engineering.

He is an Academician of the Chinese Academy of Engineering.